

Progetto di Elementi di Intelligenza Artificiale

Corso: Elementi di Intelligenza Artificiale

Docente: Prof. Sperli Giancarlo

Studente: Leonardo Vernieri

Matricola: N46007473

Anno Accademico: 2025/2026

Indice

1. Obiettivo del Progetto
 2. Introduzione alla Teoria dei Giochi
 3. Il Gioco del Nim
 4. La Nim-sum e il Teorema di Sprague-Grundy
 5. Algoritmo Minimax
 6. Alpha-Beta Pruning
 7. Descrizione dell'Implementazione
 8. Risultati Sperimentali
 9. Conclusioni
-

1. Obiettivo del Progetto

L'obiettivo del progetto è la realizzazione di un software che applica concetti fondamentali della **Teoria dei Giochi** al gioco combinatorio del **Nim**. In particolare, il software implementa tre agenti con strategie di gioco diverse e ne confronta le prestazioni tramite tornei automatici:

- **RandomAgent:** agente baseline che seleziona le mosse in modo casuale.
- **XORAgent:** agente che applica la strategia ottima analitica derivata dal teorema di Sprague-Grundy, basata sulla Nim-sum (XOR bit a bit delle pile).
- **MinimaxAgent:** agente che esplora l'albero di gioco tramite l'algoritmo Minimax, con supporto opzionale all'Alpha-Beta Pruning.

Il software è sviluppato in **Python** e strutturato in quattro moduli distinti (`nim.py` , `agents.py` , `experiments.py` , `main.py`), seguendo il principio di responsabilità singola: ogni modulo ha un compito ben definito e indipendente dagli altri.

L'utilizzo del software è consigliato per istanze di gioco deterministiche e a informazione perfetta, caratteristiche proprie del Nim nella sua variante classica.

2. Introduzione alla Teoria dei Giochi

La **Teoria dei Giochi** è una branca della matematica e dell'economia che studia i modelli di interazione strategica tra agenti razionali. Fu formalizzata da John von Neumann e Oskar Morgenstern nel 1944 con la pubblicazione di "Theory of Games and Economic Behavior", e successivamente estesa da John Nash con il concetto di **equilibrio di Nash**.

2.1 Classificazione dei giochi

I giochi studiati dalla teoria possono essere classificati secondo diverse dimensioni:

Numero di giocatori: si distingue tra giochi a due giocatori (come il Nim) e giochi a N giocatori. I giochi a due giocatori sono i più studiati perché ammettono una trattazione matematica più rigorosa.

Somma del gioco: un gioco si dice **a somma zero** quando il guadagno di un giocatore corrisponde esattamente alla perdita dell'altro. In questi giochi gli interessi dei due giocatori sono perfettamente contrapposti. Il Nim è un gioco a somma zero: c'è sempre un vincitore e un perdente, senza possibilità di pareggio o di beneficio reciproco.

Informazione: un gioco è a **informazione perfetta** quando entrambi i giocatori conoscono in ogni momento l'intero stato del gioco, senza elementi nascosti o casuali. Il Nim rientra in questa categoria: le pile sono visibili a entrambi i giocatori e non esiste alcun elemento di casualità. Altri esempi di giochi a informazione perfetta sono gli scacchi e la dama; esempi di giochi a informazione imperfetta sono il poker e il Monopoly.

Determinismo: un gioco è **deterministico** quando le azioni dei giocatori producono sempre lo stesso esito, senza componenti aleatorie. Il Nim è deterministico: rimuovere un certo numero di oggetti da una pila ha sempre un unico risultato prevedibile.

Il Nim soddisfa tutte e tre le caratteristiche sopra descritte: è un gioco a **due giocatori, a somma zero, a informazione perfetta e deterministico**. Questa combinazione di proprietà lo rende un caso di studio ideale per applicare i metodi della teoria dei giochi, e in particolare l'algoritmo Minimax.

2.2 Strategie e soluzioni

In un gioco a due giocatori a somma zero e informazione perfetta, si definisce **strategia** una funzione che, dato lo stato del gioco, indica quale mossa compiere. Una strategia si dice **ottima** se garantisce il miglior risultato possibile indipendentemente dal comportamento dell'avversario.

Il teorema fondamentale di von Neumann (minimax theorem, 1928) afferma che in ogni gioco finito a due giocatori a somma zero esiste sempre almeno una coppia di strategie ottime, una per ciascun giocatore. Questo risultato garantisce che il Nim, essendo un gioco finito di questo tipo, ammette una soluzione teorica esatta — e questa soluzione è esattamente ciò che l'XORAgent implementa.

3. Il Gioco del Nim

3.1 Regole

Il Nim è un gioco combinatorio a due giocatori. Le regole sono semplici:

1. Si dispongono N pile di oggetti (fiammiferi, monete, sassi, ecc.). Ogni pila contiene un numero arbitrario di oggetti.
2. I due giocatori si alternano. A ogni turno, il giocatore attivo sceglie **una** pila e rimuove da essa **almeno un oggetto** (fino a rimuoverli tutti).
3. **Vince chi prende l'ultimo oggetto** (variante "normale"). Esiste anche la variante "Misère", in cui perde chi prende l'ultimo oggetto, ma non è oggetto di questo progetto.

Il gioco termina quando tutte le pile sono vuote. Non esiste la possibilità di pareggio.

3.2 Rappresentazione dello spazio degli stati

Lo spazio degli stati del Nim può essere modellato come un **grafo orientato aciclico** (DAG): i nodi rappresentano le configurazioni delle pile, e gli archi rappresentano le mosse legali. Poiché ogni mossa riduce strettamente il

numero totale di oggetti, il grafo non ha cicli e il gioco termina sempre in un numero finito di mosse.

Ogni stato è identificato univocamente dalla tupla dei valori delle pile e dall'indicatore del giocatore di turno. Per esempio, lo stato $([3, 5, 7], \text{giocatore } 0)$ rappresenta la configurazione con tre pile di 3, 5 e 7 oggetti in cui è il turno del primo giocatore.

Il numero di stati possibili cresce rapidamente con la dimensione delle pile: per una configurazione $[n_1, n_2, \dots, n_k]$ il numero massimo di stati è dell'ordine di $(n_1+1) \times (n_2+1) \times \dots \times (n_k+1)$. Per la configurazione classica $[3, 5, 7]$ si hanno al più $4 \times 6 \times 8 = 192$ stati distinti, un numero perfettamente gestibile dall'algoritmo Minimax. Per configurazioni più grandi, come $[4, 5, 6, 7]$, il numero di stati aumenta significativamente e rende necessario l'utilizzo di un limite di profondità.

3.3 Configurazioni di partenza e posizioni

Nel Nim si distinguono due tipi di posizione:

- **P-position** (Previous player wins): la posizione è perdente per chi deve muovere. Qualunque mossa lascia l'avversario in una posizione favorevole.
- **N-position** (Next player wins): la posizione è vincente per chi deve muovere. Esiste almeno una mossa che porta l'avversario in una P-position.

La classificazione di una posizione in P o N dipende dalla **Nim-sum**, come illustrato nella sezione successiva.

4. La Nim-sum e il Teorema di Sprague-Grundy

4.1 Definizione di Nim-sum

La Nim-sum di una configurazione di pile è definita come lo **XOR bit a bit** di tutti i valori delle pile:

$$\text{nim-sum} = p_1 \oplus p_2 \oplus \dots \oplus p_n$$

dove $\oplus$ indica l'operazione XOR (or esclusivo) e p_i è il numero di oggetti nella pila i .

L'operazione XOR agisce bit per bit su due numeri binari secondo la seguente tabella:

Bit A	Bit B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Lo XOR restituisce 1 quando i due bit sono diversi, 0 quando sono uguali. È importante notare che XOR differisce dall'OR ordinario proprio nell'ultimo caso:

$1 \text{ OR } 1 = 1$, ma $1 \text{ XOR } 1 = 0$.

Esempio con la configurazione classica $[3, 5, 7]$:

```

3 = 0 1 1
5 = 1 0 1
7 = 1 1 1
-----
XOR= 0 0 1 = 1

```

La nim-sum vale 1, che è diversa da zero: si tratta dunque di una **N-position**, favorevole a chi inizia.

Esempio con la configurazione $[1, 2, 3]$:

```

1 = 0 1
2 = 1 0
3 = 1 1
-----
XOR= 0 0 = 0

```

La nim-sum vale 0: si tratta di una **P-position**, svantaggiosa per chi inizia.

4.2 Teorema di Sprague-Grundy

Il risultato matematico fondamentale che governa il Nim è il **teorema di Sprague-Grundy**, dimostrato indipendentemente da Roland Sprague (1935) e Patrick Grundy (1939):

Teorema: Una posizione del Nim è una P-position (perdente per chi muove) se e solo se la sua nim-sum è uguale a zero. È una N-position (vincente per chi muove) se e solo se la sua nim-sum è diversa da zero.

Il teorema implica due conseguenze operative fondamentali:

1. **Da una P-position (nim-sum = 0):** qualunque mossa si compia, si porta l'avversario in una N-position. Non esiste alcuna mossa che lasci l'avversario in P-position.
2. **Da una N-position (nim-sum $\neq 0$):** esiste sempre almeno una mossa che porta l'avversario in P-position (nim-sum = 0). Questa mossa è la mossa ottima.

4.3 Come trovare la mossa ottima

Partendo da una N-position con nim-sum $s \neq 0$, la mossa ottima si trova come segue:

1. Per ogni pila p_i , calcola $t_i = p_i \oplus s$.
2. Se $t_i < p_i$, allora rimuovere $p_i - t_i$ oggetti dalla pila i porta la nim-sum globale a zero.

Dimostrazione: dopo la mossa, la pila i vale $t_i = p_i \oplus s$. La nuova nim-sum è:

$$t_i \oplus \bigoplus_{j \neq i} p_j = (p_i \oplus s) \oplus \bigoplus_{j \neq i} p_j = s \oplus \bigoplus_j p_j = s \oplus s = 0$$

dove si è usato il fatto che $A \oplus A = 0$ per qualsiasi A .

Esempio con [3, 5, 7] e nim-sum = 1:

- Pila 0: $t_0 = 3 \oplus 1 = 2$. Poiché $2 < 3$, si rimuove $3 - 2 = 1$ oggetto dalla pila 0.
- Verifica: $2 \oplus 5 \oplus 7 = 0$. ✓

5. Algoritmo Minimax

5.1 Descrizione

L'algoritmo **Minimax** è un metodo di ricerca ad albero per trovare la strategia ottima in giochi a due giocatori, a somma zero, a informazione perfetta e deterministici. Fu introdotto da John von Neumann nel 1928 come parte del minimax theorem.

L'algoritmo assume l'esistenza di due giocatori:

- **MAX:** il giocatore che l'agente rappresenta. Vuole *massimizzare* il valore del gioco.

- **MIN:** l'avversario. Vuole *minimizzare* il valore del gioco.

Ad ogni stato terminale viene assegnato un valore numerico:

- $+1$ se ha vinto MAX
- -1 se ha vinto MIN

L'algoritmo esplora ricorsivamente l'albero di gioco a partire dalla radice (stato corrente) fino alle foglie (stati terminali), e risale assegnando valori agli stati intermedi secondo la regola:

- **Nei nodi MAX** (turno di MAX): il valore dello stato è il **massimo** dei valori dei figli.
- **Nei nodi MIN** (turno di MIN): il valore dello stato è il **minimo** dei valori dei figli.

La mossa scelta da MAX nella radice è quella che porta allo stato figlio con valore massimo.

5.2 Pseudocodice

```
funzione MINIMAX(stato, è_max):
    se stato è terminale:
        restituisci valore(stato)    // +1, -1

    se è_max:
        valore =  $-\infty$ 
        per ogni mossa in mosse_legali(stato):
            figlio = applica_mossa(stato, mossa)
            valore = max(valore, MINIMAX(figlio, falso))
        restituisci valore
    altrimenti:
        valore =  $+\infty$ 
        per ogni mossa in mosse_legali(stato):
            figlio = applica_mossa(stato, mossa)
            valore = min(valore, MINIMAX(figlio, vero))
        restituisci valore
```

5.3 Proprietà

- **Completezza:** Minimax è completo — trova sempre una soluzione se esiste, esplorando l'intero albero di gioco.
- **Ottimalità:** la mossa trovata è ottima contro un avversario che gioca in modo ottimo.

- **Complessità temporale:** $O(b^m)$, dove b è il branching factor (numero medio di mosse legali) e m è la profondità massima dell'albero.
- **Complessità spaziale:** $O(bm)$, proporzionale alla profondità, poiché si esplora un ramo alla volta.

Nel caso del Nim, b varia in base alla configurazione delle pile. Per [3, 5, 7] si hanno 15 mosse nel primo turno, e l'albero completo può arrivare a decine di migliaia di nodi.

6. Alpha-Beta Pruning

6.1 Motivazione

La complessità esponenziale $O(b^m)$ del Minimax lo rende impraticabile per alberi molto profondi o con branching factor elevato. L'**Alpha-Beta Pruning** è un'ottimizzazione che riduce drasticamente il numero di nodi esplorati mantenendo **esattamente la stessa decisione** del Minimax classico.

6.2 Funzionamento

L'Alpha-Beta Pruning mantiene due valori durante la ricerca:

- **alpha** (α): il miglior valore che MAX può garantirsi nel percorso corrente dall'alto. Inizia a $-\infty$.
- **beta** (β): il miglior valore che MIN può garantirsi nel percorso corrente dall'alto. Inizia a $+\infty$.

La regola di potatura è:

- **Potatura beta** (in un nodo MAX): se il valore corrente supera β , MIN non sceglierà mai questo nodo (ha già trovato un'alternativa migliore). Si interrompe l'esplorazione dei figli rimanenti.
- **Potatura alpha** (in un nodo MIN): se il valore corrente scende sotto α , MAX non sceglierà mai questo nodo. Si interrompe l'esplorazione dei figli rimanenti.

6.3 Pseudocodice

```
funzione ALPHABETA(stato, alpha, beta, è_max):
    se stato è terminale:
        restituisci valore(stato)
```



```

se è_max:
    valore = -∞
    per ogni mossa in mosse_legalì(stato):
        figlio = applica_mossa(stato, mossa)
        valore = max(valore, ALPHABETA(figlio, alpha, beta, falso))
        alpha = max(alpha, valore)
        se beta ≤ alpha:
            interrompi // potatura beta
    restituisci valore
altrimenti:
    valore = +∞
    per ogni mossa in mosse_legalì(stato):
        figlio = applica_mossa(stato, mossa)
        valore = min(valore, ALPHABETA(figlio, alpha, beta, vero))
        beta = min(beta, valore)
        se beta ≤ alpha:
            interrompi // potatura alpha
    restituisci valore

```

6.4 Efficienza

Nel caso migliore (mosse ordinate ottimalmente), l'Alpha-Beta Pruning riduce la complessità temporale a $O(b^{m/2})$: a parità di tempo, può esplorare un albero il doppio più profondo rispetto al Minimax classico. Nel caso medio con ordine casuale delle mosse, la complessità è circa $O(b^{3m/4})$.

7. Descrizione dell'Implementazione

Il progetto è organizzato in quattro moduli Python, ciascuno con una responsabilità specifica.

7.1 `nim.py` — Rappresentazione dello stato

Il modulo definisce la classe `NimState`, che rappresenta uno snapshot completo del gioco: la configurazione delle pile e il giocatore di turno.

I metodi principali sono:

- `is_terminal()`: restituisce `True` se tutte le pile sono vuote, usando `all(p == 0 for p in self.piles)`.
- `get_legal_moves()`: genera tutte le mosse legali come lista di coppie `(indice_pila, quantità)`. Per una configurazione `[n1, ..., nk]` genera $\sum_i n_i$ mosse.

- `apply_move(move)` : applica una mossa e restituisce un **nuovo** oggetto `NimState` senza modificare quello corrente (approccio immutabile, necessario per l'esplorazione dell'albero Minimax).
- `nim_sum()` : calcola lo XOR bit a bit di tutte le pile tramite accumulazione: `result ^= pile` per ogni pila.

7.2 `agents.py` — Agenti di gioco

Il modulo implementa tre agenti, tutti con la stessa interfaccia (`choose_move(state)`), che permette di usarli in modo intercambiabile nel motore di gioco (polimorfismo).

RandomAgent: seleziona una mossa a caso con

`random.choice(state.get_legal_moves())`. Serve come baseline inferiore.

XORAgent: implementa la strategia ottima di Sprague-Grundy. Calcola la nim-sum e, se è diversa da zero, trova la mossa vincente iterando sulle pile: per ogni pila p_i calcola `target = pile ^ nim_sum` e, se `target < pile`, rimuove `pile - target` oggetti. Se la nim-sum è zero (P-position), gioca a caso.

MinimaxAgent: implementa Minimax con Alpha-Beta Pruning opzionale. Il parametro `max_player_id` indica quale giocatore rappresenta MAX. Il parametro `depth_limit` permette di limitare la profondità di esplorazione per configurazioni grandi. Il contatore `nodes_explored` viene incrementato a ogni visita di un nodo e resettato prima di ogni decisione, per misurare il lavoro computazionale dell'algoritmo.

Quando il limite di profondità viene raggiunto senza aver trovato uno stato terminale, si applica una **funzione euristica** basata sulla nim-sum: una posizione con nim-sum diversa da zero è stimata leggermente favorevole per chi deve muovere. I valori euristici `±0.1` sono volutamente distanti dai valori certi `±1.0` degli stati terminali, così il Minimax preferisce sempre una vittoria certa a una vittoria stimata.

7.3 `experiments.py` — Motore dei tornei

Il modulo gestisce la raccolta sistematica dei risultati sperimentali.

- `run_game()` : esegue una singola partita misurando il tempo con `time.perf_counter()` (più preciso di `time.time()` per intervalli brevi) e raccogliendo il numero di mosse e i nodi esplorati.

- `run_tournament()` : esegue N partite alternando chi inizia (`alternate_start=True`). L'alternanza è fondamentale per la correttezza statistica: in una N-position, chi inizia ha un vantaggio strutturale, e non alternare favorirebbe sistematicamente un agente indipendentemente dalla qualità della sua strategia.
- `analyze_nim_sum()` : visualizza la decomposizione XOR bit a bit di una configurazione usando la formattazione `{p:08b}` di Python (rappresentazione binaria su 8 cifre), utile per verificare manualmente i calcoli teorici.

7.4 `main.py` — Punto d'ingresso

Il modulo orchestra le tre modalità d'uso del software: partita interattiva, esperimenti automatici, analisi della nim-sum. Definisce anche `HumanAgent`, che implementa la stessa interfaccia degli agenti algoritmici ma legge la mossa dall'input dell'utente, dimostrando il pattern polimorfico in modo pratico.

8. Risultati Sperimentali

I test sono stati effettuati su quattro configurazioni di pile, scelte per coprire casi strutturalmente diversi:

Configurazione	Nim-sum	Tipo	Depth limit
<code>[1, 2, 3]</code>	0	P-position	completo
<code>[3, 5, 7]</code>	1	N-position	d=5
<code>[2, 4, 6]</code>	0	P-position	d=5
<code>[4, 5, 6, 7]</code>	0	P-position	d=4

Per ogni configurazione sono stati eseguiti tornei da 200 partite tra le coppie: Minimax+AB vs XOR, Minimax+AB vs Random, XOR vs Random, Minimax+AB vs Minimax (senza pruning) — esclusa quest'ultima per `[4,5,6,7]` per ragioni computazionali.

8.1 Configurazione `[1, 2, 3]` — P-position (nim-sum = 0)

Scontro	Win% A1	Win% A2	Mosse medie	Tempo medio (ms)
Minimax+AB vs XOR	0%	100%	5.2	0.577

Scontro	Win% A1	Win% A2	Mosse medie	Tempo medio (ms)
Minimax+AB vs Random	67.5%	32.5%	4.4	0.483
XOR vs Random	89.5%	10.5%	4.3	0.013
Minimax+AB vs Minimax	50%	50%	6.0	1.145

Osservazioni:

Il risultato Minimax+AB 0% vs XOR 100% è atteso e teoricamente corretto: entrambi giocano in modo ottimo, e in una P-position chi inizia perde con gioco perfetto da entrambe le parti. Il Minimax riconosce che non esiste mossa vincente e viene battuto dall'XORAgent che gioca altrettanto ottimamente.

Il risultato Minimax+AB 67.5% vs Random dimostra che anche in una P-position l'agente Minimax riesce comunque a vincere circa 2 volte su 3 contro un avversario casuale: il Random commette errori che Minimax sfrutta.

Il risultato XOR 89.5% vs Random 10.5% conferma la superiorità della strategia analitica: anche partendo svantaggiato in una P-position, l'XORAgent riesce quasi sempre a convertire gli errori del Random in vittorie.

8.2 Configurazione [3, 5, 7] — N-position (nim-sum = 1)

Scontro	Win% A1	Win% A2	Mosse medie	Tempo medio (ms)
Minimax+AB vs XOR	50%	50%	7.3	16.227
Minimax+AB vs Random	73.0%	27.0%	7.4	17.731
XOR vs Random	99.5%	0.5%	6.9	0.015
Minimax+AB vs Minimax	50%	50%	12.0	135.865

Osservazioni:

Il risultato XOR 99.5% vs Random 0.5% conferma che in una N-position l'XORAgent è quasi imbattibile contro un avversario casuale: partendo con vantaggio strutturale e giocando in modo ottimo, lascia rarissimi margini di errore.

Il risultato Minimax+AB (d=5) 50% vs XOR è spiegabile dal limite di profondità: con `depth_limit=5`, il Minimax non esplora l'albero fino alle foglie e si affida all'euristica, perdendo parte dell'ottimalità. Un Minimax completo raggiungerebbe risultati analoghi all'XOR.

8.3 Configurazione [2, 4, 6] — P-position (nim-sum = 0)

Scontro	Win% A1	Win% A2	Mosse medie	Tempo medio (ms)
Minimax+AB vs XOR	0%	100%	9.0	11.364
Minimax+AB vs Random	68.0%	32.0%	6.5	11.491
XOR vs Random	99.0%	1.0%	5.8	0.027
Minimax+AB vs Minimax	50%	50%	11.0	40.924

Osservazioni:

Il risultato Minimax+AB 0% vs XOR 100% replica il risultato di [1,2,3]: entrambe sono P-position e con gioco ottimo da entrambe le parti chi inizia perde sempre. Il risultato è coerente tra configurazioni strutturalmente equivalenti.

Il risultato XOR 99% vs Random conferma la robustezza della strategia analitica su P-position con pile più grandi, dove il numero di mosse possibili è maggiore.

8.4 Configurazione [4, 5, 6, 7] — P-position (nim-sum = 0)

Scontro	Win% A1	Win% A2	Mosse medie	Tempo medio (ms)
Minimax+AB vs XOR	0%	100%	14.7	30.930
Minimax+AB vs Random	66.0%	34.0%	10.5	21.199
XOR vs Random	100%	0%	9.6	0.047

Osservazioni:

Il risultato XOR 100% vs Random su 4 pile: la strategia analitica è perfetta. L'XORAgent non perde mai contro un avversario casuale, indipendentemente dal numero di pile.

Il risultato Minimax+AB (d=4) 0% vs XOR conferma che con un limite di profondità ridotto e una configurazione da 4 pile il Minimax non riesce a trovare la mossa ottima in ogni stato, cedendo il 100% delle partite all'XORAgent che gioca perfettamente.

8.5 Confronto Minimax con e senza Alpha-Beta Pruning

Configurazione	Agente	Nodi esplorati (media)	Tempo medio (ms)
[1, 2, 3]	Minimax + AB	~4	~1.1
[1, 2, 3]	Minimax (no AB)	~1	~1.1
[3, 5, 7]	Minimax + AB	~1	~16.2
[3, 5, 7]	Minimax (no AB)	~4	~135.9
[2, 4, 6]	Minimax + AB	~4	~11.5
[2, 4, 6]	Minimax (no AB)	~1	~40.9

Nota: su configurazioni piccole come [1,2,3], l'albero di gioco è molto limitato (poche decine di nodi totali) e la differenza tra Minimax con e senza Alpha-Beta Pruning è minima. Il vantaggio dell'Alpha-Beta diventa significativo su configurazioni più grandi, come [3,5,7] dove il tempo medio passa da ~16 ms (con AB) a ~136 ms (senza AB): un fattore ~8x. La potatura elimina interi sottonodi irrilevanti riducendo i nodi esplorati di uno o più ordini di grandezza, senza alterare la decisione finale.

8.6 Analisi della Nim-sum — Esempio didattico

Di seguito è riportato l'output del software per la configurazione [3, 5, 7]:

```

=====
Analisi Nim-sum per pile = [3, 5, 7]
=====
Pila 1: 3 → 00000011
Pila 2: 5 → 00000101
Pila 3: 7 → 00000111
=====
Nim-sum: 1 → 00000001
✓ Posizione VINCENTE per chi deve muovere (N-position)
=====

```

La visualizzazione bit a bit rende immediatamente verificabile il calcolo XOR colonna per colonna, confermando che la nim-sum è 1 e che la posizione è quindi una N-position.

8.7 Riepilogo complessivo (tutte le configurazioni)

Scontro	Vittorie A1	Vittorie A2	Win% A1
Minimax+AB (completo) vs XOR	0	200	0.0%
Minimax+AB (completo) vs Random	135	65	67.5%
XOR vs Random	776	24	97.0%
Minimax+AB (completo) vs Minimax (completo)	100	100	50.0%
Minimax+AB (d=5) vs XOR	100	300	25.0%
Minimax+AB (d=5) vs Random	282	118	70.5%
Minimax+AB (d=5) vs Minimax (d=5)	200	200	50.0%
Minimax+AB (d=4) vs XOR	0	200	0.0%
Minimax+AB (d=4) vs Random	132	68	66.0%

9. Conclusioni

I risultati sperimentali confermano pienamente le previsioni teoriche derivate dal teorema di Sprague-Grundy: l'**XORAgent**, applicando la strategia analitica ottima basata sulla nim-sum, risulta imbattibile contro il **MinimaxAgent** completo nelle P-position e ottiene percentuali di vittoria vicini al 100% contro il **RandomAgent** in N-position. Il **MinimaxAgent**, pur essendo un approccio generale non specifico per il Nim, riesce a trovare strategie efficaci esplorando l'albero di gioco, con prestazioni che dipendono fortemente dalla profondità di esplorazione.

Sul piano computazionale, il confronto tra Minimax con e senza Alpha-Beta Pruning ha evidenziato come la potatura, pur non modificando la qualità della decisione, riduca il numero di nodi esplorati in modo significativo su configurazioni di dimensioni maggiori. Questo ha implicazioni pratiche rilevanti: l'Alpha-Beta Pruning permette di applicare il Minimax a giochi più complessi rimanendo nei limiti computazionali accettabili.

Sviluppi futuri potrebbero includere: l'estensione alla variante Misère del Nim (in cui perde chi prende l'ultimo oggetto, con una teoria leggermente diversa), l'implementazione di algoritmi di ricerca informata come A* per versioni del gioco con costi non uniformi, e la generalizzazione ad altri giochi combinatori

tramite la teoria completa di Grundy (Sprague-Grundy generalizzata a giochi sommabili).